
Cluster Analysis

Contents

- Cluster Analysis: Basic Concepts 
- Partitioning Methods
- Hierarchical Methods
- Density-Based Methods
- Evaluation of Clustering
- Summary

What is Cluster Analysis?

- Cluster: A collection of data objects
 - similar (or related) to one another within the same group
 - dissimilar (or unrelated) to the objects in other groups
- Cluster analysis
 - Finding similarities between data according to the characteristics found in the data and grouping similar data objects into clusters
- **Unsupervised learning**: no predefined class

Applications

- Typical applications

A) As a **stand-alone tool** to get insight into data distribution

- Information retrieval: document clustering
- City-planning: Identifying groups of houses according to their house type, value, and geographical location
- Climate: understanding earth climate, find patterns of atmospheric and ocean

B) As a **preprocessing step** for other algorithms

- Summarization: Consider representatives of all clusters
- Outlier detection: Outliers are often viewed as those “far away” from any cluster

Quality: What Is Good Clustering?

- A good clustering method will produce high quality clusters
 - high intra-class similarity: **cohesive** within clusters
 - low inter-class similarity: **distinctive** between clusters
- The quality of a clustering method depends on
 - the similarity measure used by the method
 - its implementation, and
 - Its ability to discover some or all of the hidden patterns

Measure the Quality of Clustering

- Dissimilarity/Similarity metric
 - Similarity is expressed in terms of a distance function, typically metric: $d(i, j)$
 - The definitions of distance functions are usually rather different for interval-scaled, boolean, nominal / categorical, ordinal, and ratio-scaled variables
 - Weights should be associated with different variables based on applications and data semantics

Interval-scaled variables

- These variables are continuous measurements of a roughly linear scale. Typical examples include weight, height, latitude, longitude, and weather temperature.
- The measurement unit used can affect the clustering analysis. For ex., changing measurement units from meters to inches for height, or from kilograms to pounds for weight, may lead to a very different clustering structure.
- To help avoid dependence on the choice of measurement units, the data should be standardized by techniques like Min-Max, Z-Score, decimal scaling etc.
- The dissimilarity (or similarity) between the objects described by interval-scaled variables is typically computed based on the distance between each pair of objects.

Interval-scaled variables

- The most popular distance measure is Euclidean distance
- Let, $i = (x_{i1}, x_{i2}, \dots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \dots, x_{jp})$ are two p-dimensional data objects.
- The Minkowski distance $d(i,j)$ is:

$$d(i,j) = (|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)^{\frac{1}{q}}, \quad \text{where } q \text{ is a positive integer}$$

- If $q=1$, then $d(i,j)$ represents the Manhattan distance
- If $q=2$, then $d(i,j)$ represents the Euclidean distance
- Both the Manhattan distance and Euclidean distance satisfy the mathematical requirements of a distance function.

Mathematical requirements of a distance function

- i) $d(i, j) \geq 0$; distance is a nonnegative number.
- ii) $d(i, i) = 0$; distance of an object to itself is 0.
- iii) $d(i, j) = d(j, i)$; distance is a symmetric function.
- iv) $d(i, j) \leq d(i, h) + d(h, j)$; going directly from object i to object j in space is no more than making a detour over any other object h (triangular inequality)

Interval-scaled variables

- If each variable is assigned a weight according to its perceived importance, the weighted Euclidean distance can be computed as

$$d(i, j) = (w_1 |x_{i1} - x_{j1}|^2 + w_2 |x_{i2} - x_{j2}|^2 + \dots + w_p |x_{ip} - x_{jp}|^2)^{\frac{1}{2}}$$

Binary Variables

- A binary variable has only two states: 0 or 1, where 0 means that the variable is absent, and 1 means that it is present.
- Example, smoker, where its value 1 indicates that the patient smokes, while the value 0 means the patient does not smoke.
- Out of p dimensions, for objects i , and object j , let
 - i) q is the number of variables that equal to 1 for both i , and j
 - ii) r is the number of variables that equal to 1 for i , and 0 for j
 - iii) s is the number of variables that equal to 0 for i , and 1 for j
 - iv) t is the number of variables that equal to 0 for both i , and j
- Therefore, $p = q + r + s + t$

Symmetric/Asymmetric Binary Variables

A. Symmetric Binary Variables

- Here, both the states (i.e., 0 and 1) are equally valuable and carry the same weight.
- Here, dissimilarity is:
$$d(i, j) = \frac{r + s}{q + r + s + t}$$

B. Asymmetric Binary Variables

- Here, both the states (i.e., 0 and 1) are not equally important (e.g., positive and negative outcomes of a disease test)
- By convention, the most important outcome is represented by 1, and the other outcome as 0
- Here, dissimilarity is:
$$d(i, j) = \frac{r + s}{q + r + s} \text{ (known as Jaccard coefficient)}$$

Nominal variables

- It is a generalization of the binary variable, where the variable can take on more than two states. Ex., map_color, that may have many states, like red, yellow, green, pink, and blue
- Let the number of states of a nominal variable be M. The states can be denoted letters, symbols, or a set of integers, such as 1, 2, ..., M, note that such integers are used just for data handling and do not represent any specific ordering.
- Here, dissimilarity is: $d(i, j) = \frac{p - m}{p}$, where m is the number of matches (i.e., the number of variables for which i and j are in the same state), and p is the total number of variables.
- Weights can be assigned to increase the effect of m or to assign greater weight to the matches in variables having a larger number of states.

Ordinal variables

- A **ordinal variable** resembles a nominal variable, except that the M states of the ordinal value are ordered in a meaningful sequence. Ex., Professional ranks are often enumerated in a sequential order, such as assistant, associate, and full.
- They are very useful for registering subjective assessments of qualities that cannot be measured objectively.
- The ordinal variables may also be obtained from the discretization of interval-scaled quantities by splitting the value range into a finite number of classes (use discretization algorithm).
- The values of an order variable can be mapped to ranks.
- Suppose, an order variable f has M_f states. These order states define the ranking $1, 2, \dots, M_f$.
- How are the order variables handled for measuring similarities ?

How are the order variables handled for measuring similarities ?

- Treatment is quite similar to that of interval-scaled variables when computing the dissimilarity between objects. The dissimilarity computation w.r.t. f (order variable) involves the following steps:
 - i) The value of f for the i -th object is x_{if} , and f has M_f order states, representing the ranking 1, 2, ..., M_f . Replace each x_{if} by its corresponding rank, $r_{if} \in \{1, 2, \dots, M_f\}$.
 - ii) Since each ordinal variable can have a different number of states, it is often necessary to map the range of each variable onto $[0.0, 1.0]$ so that each variable has equal weight. This can be achieved by replacing the rank r_{if} of the f -th variable in the i -th object by $z_{if} = \frac{r_{if} - 1}{M_f - 1}$
 - iii) Dissimilarity can then be computed using any of the distance measure techniques, where z_{if} is the f -value of the i -th object.

Ratio-Scaled variables

- A ratio-scaled variable makes a positive measurement on a nonlinear scale, such as an exponential scale, approximately following the formula Ae^{Bt} or Ae^{-Bt} , where A and B are positive constants.
- Ex., the growth of a bacteria population, or the decay of a radioactive element.
- **How to compute the dissimilarity?** Following 3 different ways:
 - (i) Treat ratio-scaled variables like interval-scaled variables. This, however, is not usually a good choice since it is likely that the scale may be distorted.
 - (ii) Apply logarithmic transformation to a ratio-scaled variable f having value x_{if} for object i by using the formula $y_{if} = \log(x_{if})$. Then the y_{if} values can be treated as interval-valued. Notice that for some ratio-scaled variables, log-log or other transformations may be applied, depending on the definition and application.
 - (iii) Treat x_{if} as continuous ordinal data and treat their ranks as interval-valued.
- The latter two methods are the most effective, although the choice of method used may be dependent on the given application.

Variables of Mixed Types

- In many real databases, objects are described by a mixture of variable types. In general, a database can contain all of the six variable types discussed earlier.
- How can we compute the dissimilarity between objects with mixed type variables?
 - i) Combine the different variables into a single dissimilarity matrix, bringing all the meaningful variables onto a common scale of the interval [0.0, 1.0].
 - ii) Suppose that the data set contains p variables of mixed type. The dissimilarity $d(i,j)$ between objects i and j is defined as:
 - $d(i,j) = \frac{\sum_{f=1}^p \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^p \delta_{ij}^{(f)}}$, where the indicator $\delta_{ij}^{(f)} = 0$ if either (1) x_{if} or x_{jf} is missing, or (2) $x_{if} = x_{jf} = 0$ and variable f is asymmetric binary; otherwise $\delta_{ij}^{(f)} = 1$

Variables of Mixed Types

$$d(i, j) = \frac{\sum_{f=1}^p \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^p \delta_{ij}^{(f)}}$$

- The dissimilarity $d_{ij}^{(f)}$ is computed as follows:
 - i) If f is binary or nominal: $d_{ij}^{(f)} = 0$ if $x_{if} = x_{jf}$; otherwise $d_{ij}^{(f)} = 1$
 - ii) If f is interval-based: $d_{ij}^{(f)} = \frac{|x_{if} - x_{jf}|}{\max_h x_{hf} - \min_h x_{hf}}$, where h runs over all nonmissing objects for variable f .
 - iii) If f is ordinal or ratio-scaled: Computes the ranks r_{if} and $z_{if} = \frac{r_{if} - 1}{M_f - 1}$, and treat z_{if} as interval scaled.
- Thus the dissimilarity between objects can be computed even when the variables describing the objects are of different types.

Major Clustering Approaches (I)

- Partitioning approach:
 - Construct various partitions and then evaluate them by some criterion, e.g., minimizing the sum of square errors
 - Typical methods: k-means, k-medoids, CLARANS
- Density-based approach:
 - Based on connectivity and density functions
 - Typical methods: DBSACN, OPTICS, DenClue
- Hierarchical approach:
 - Create a hierarchical decomposition of the set of data (or objects) using some criterion
 - Typical methods: Agnes, Diana, BIRCH, CAMELEON

Major Clustering Approaches (II)

- Grid-based approach:
 - based on a multiple-level granularity structure
 - Typical methods: STING, WaveCluster, CLIQUE
- Model-based approach :
 - A model is hypothesized for each of the clusters and tries to find the best fit of that model to each other
 - Typical methods: EM, SOM, COBWEB

Cluster Analysis

- Cluster Analysis: Basic Concepts
- Partitioning Methods 
- Hierarchical Methods
- Density-Based Methods
- Evaluation of Clustering
- Summary

Partitioning Algorithms: Basic Concept

- Partitioning method: Partitioning a database ***D*** of ***n*** objects into a set of ***k*** clusters, such that the sum of squared distances is minimized (where c_i is the centroid or medoid of cluster C_i)
$$E = \sum_{i=1}^k \sum_{p \in C_i} (p - c_i)^2$$
- Given k , find a partition of k clusters that optimizes the chosen partitioning criterion
 - Global optimal: exhaustively enumerate all partitions
 - Heuristic methods: *k-means* and *k-medoids* algorithms
 - *k-means* : Each cluster is represented by the center of the cluster
 - *k-medoids* or PAM (Partition around medoids) : Each cluster is represented by one of the objects in the cluster

The *K-Means* Clustering Method

- Given k , the *k-means* algorithm is implemented in four steps:

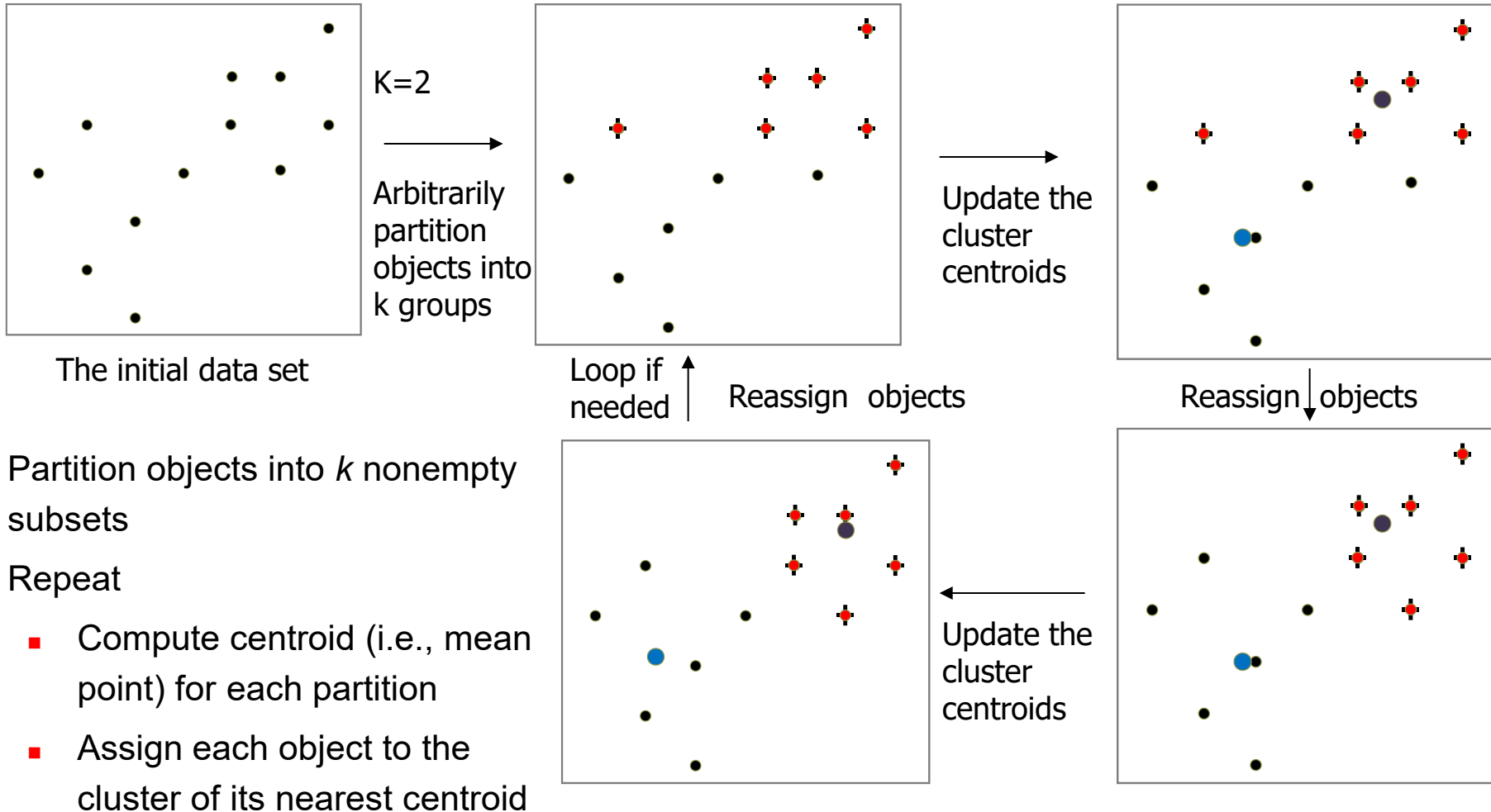
Step 1: Partition objects into k nonempty subsets

Step 2: Compute seed points as the centroids of the clusters of the current partitioning (the centroid is the center, i.e., *mean point*, of the cluster)

Step 3: Assign each object to the cluster with the nearest seed point

Step 4: Go back to Step 2, stop when the assignment does not change

An Example of *K-Means* Clustering



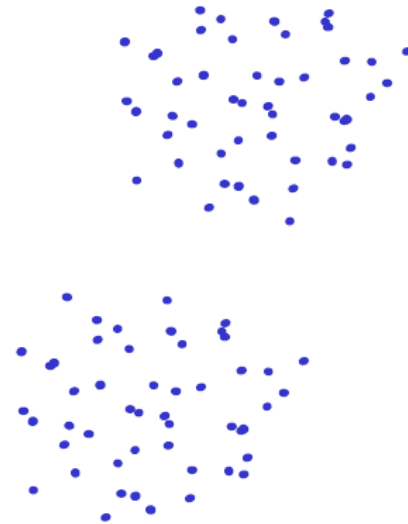
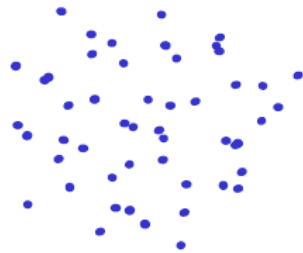
- Partition objects into k nonempty subsets
- Repeat
 - Compute centroid (i.e., mean point) for each partition
 - Assign each object to the cluster of its nearest centroid
- Until no change

Strength and weakness of *K-Means* Method

- Strength: *Efficient*: $O(tkn)$, where n is # objects, k is # clusters, and t is # iterations. Normally, $k, t \ll n$.
 - Comparing: PAM: $O(k(n-k)^2)$, CLARA: $O(ks^2 + k(n-k))$
- Weakness: Applicable only to objects in a continuous n -dimensional space
- Handling categorical data: *k-modes*
 - Replacing means of clusters with modes
 - Using new dissimilarity measures to deal with categorical objects
 - Using a frequency-based method to update modes of clusters
 - A mixture of categorical and numerical data: *k-prototype* method

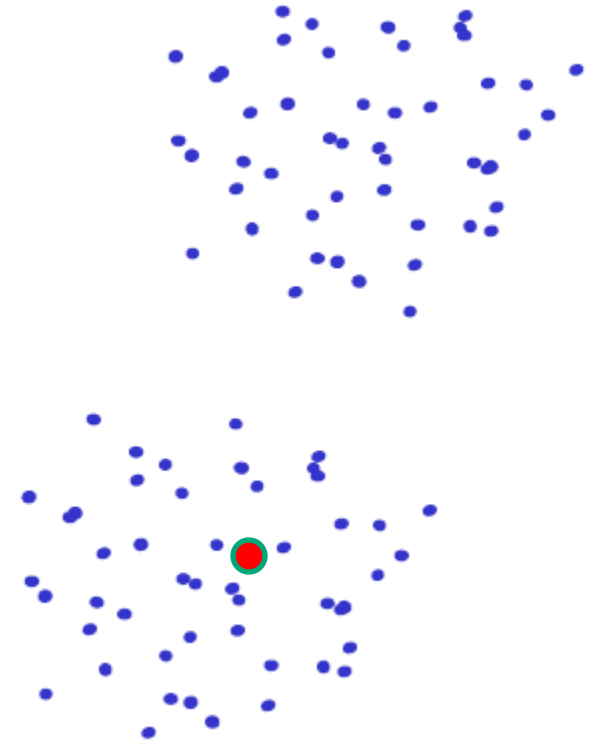
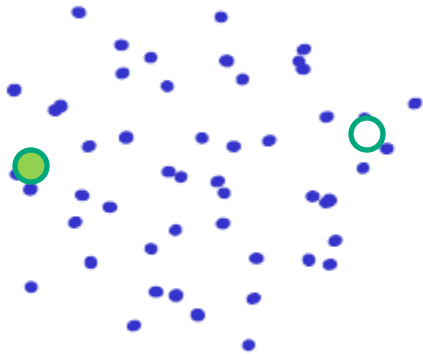
Strength and weakness of *K-Means* Method

- **Sensitivity to initial condition:** K-means is sensitive to initial conditions. The algorithm randomly initializes the cluster centroids at the beginning, and the final clustering results can vary depending on these initial positions.
- Different initializations can lead to other local optima, resulting in different clustering outcomes. This makes the K-means algorithm less reliable and reproducible.



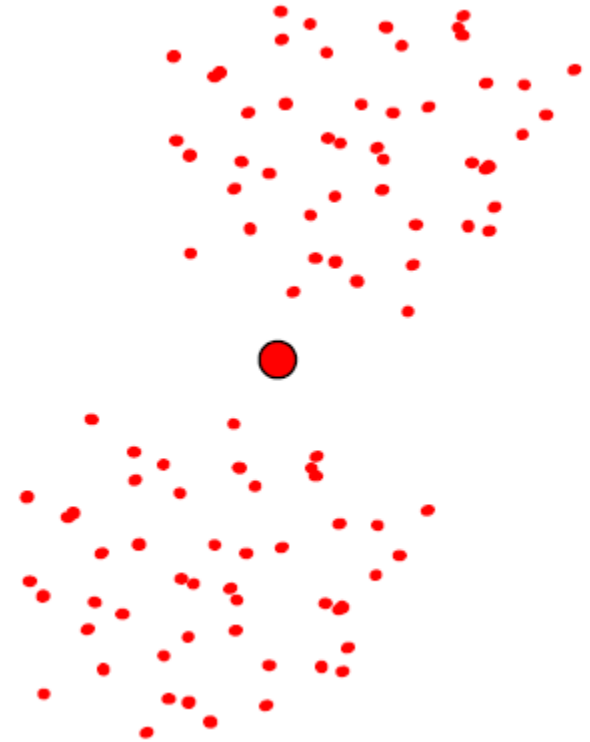
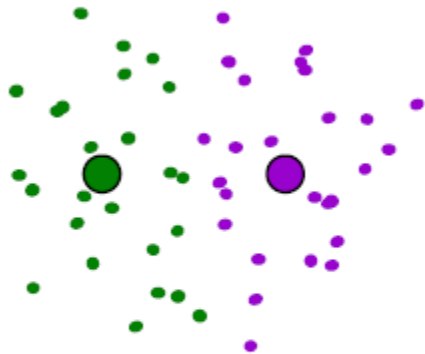
Strength and weakness of *K-Means* Method

- Dependent on initialization



Strength and weakness of *K-Means* Method

- Dependent on initialization

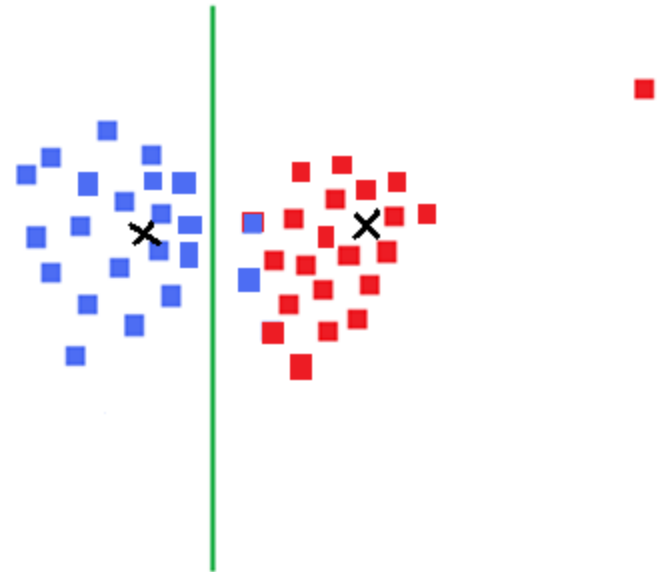


Strength and weakness of *K-Means* Method

- Dependent on initialization
 - Select random seeds with at least $D_{\min}$
 - Or, run the algorithm many times

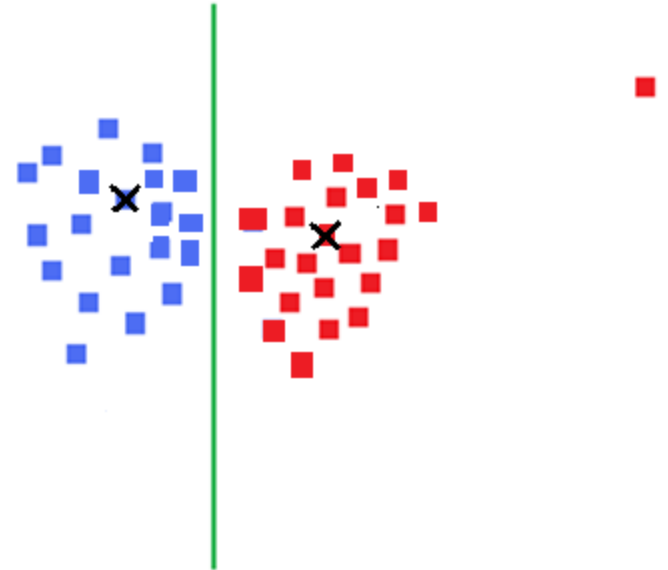
Strength and weakness of *K-Means* Method

- **Sensitive to outliers:** K-means treats all data points equally and can be sensitive to outliers, which are unusual or extreme data points.
- Outliers can distort the clustering process, causing the algorithm to create less reliable clusters.
- Handling outliers properly is important to get better results.



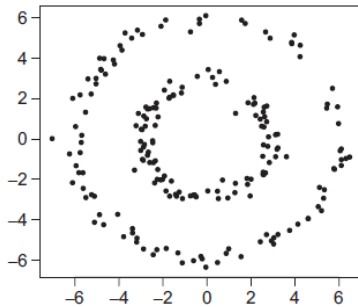
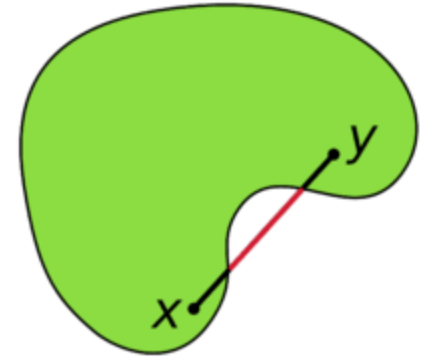
Strength and weakness of *K-Means* Method

- Sensitive to outliers
 - Use K-medoids
(which is the **most centrally located** object in a cluster)



Strength and weakness of *K-Means* Method

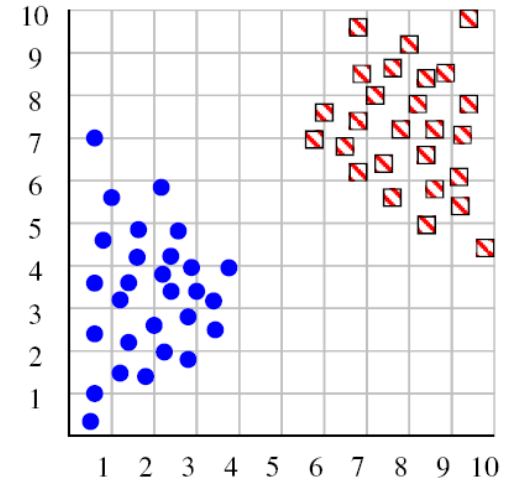
- Not suitable to discover clusters with *non-convex shapes*
- K-means assumes that clusters are round or spherical in shape. However, in real-world data, clusters can have different shapes.
- K-means may struggle to handle such irregular clusters, resulting in less accurate clusters.



- Other algorithms like DBSCAN or Gaussian Mixture Models can handle more complex cluster shapes.

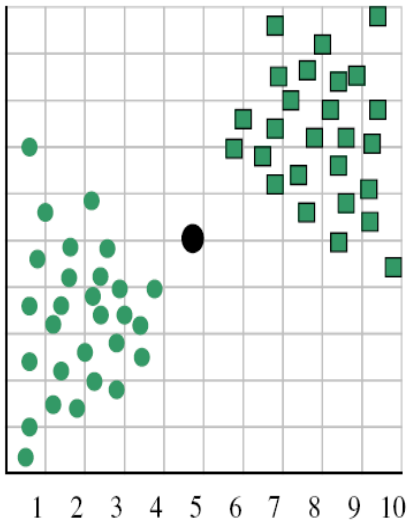
Strength and weakness of *K-Means* Method

- Deciding K : With K-means, we have to tell the algorithm how many clusters we expect in the data.
- This can be tricky, especially if we don't have prior knowledge about the data.
- Choosing the wrong number of clusters can lead to misleading results.
- Methods like the elbow finding or silhouette analysis can help estimate the appropriate number of clusters, but it's still a challenge.

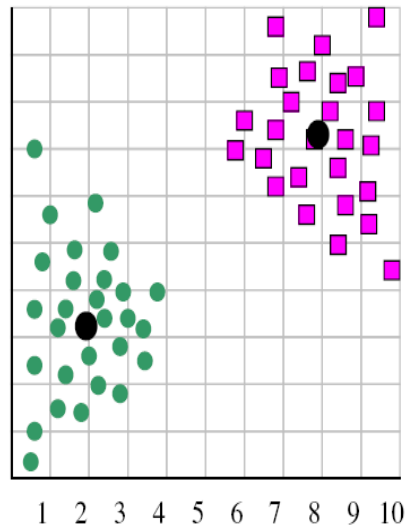


Deciding K

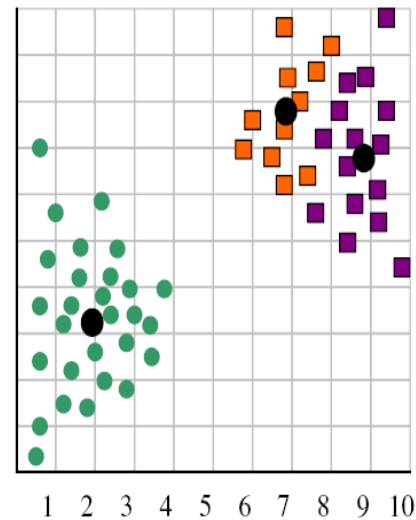
■ When $k = 1$, the objective function is 873.0



■ When $k = 2$, the objective function is 173.1

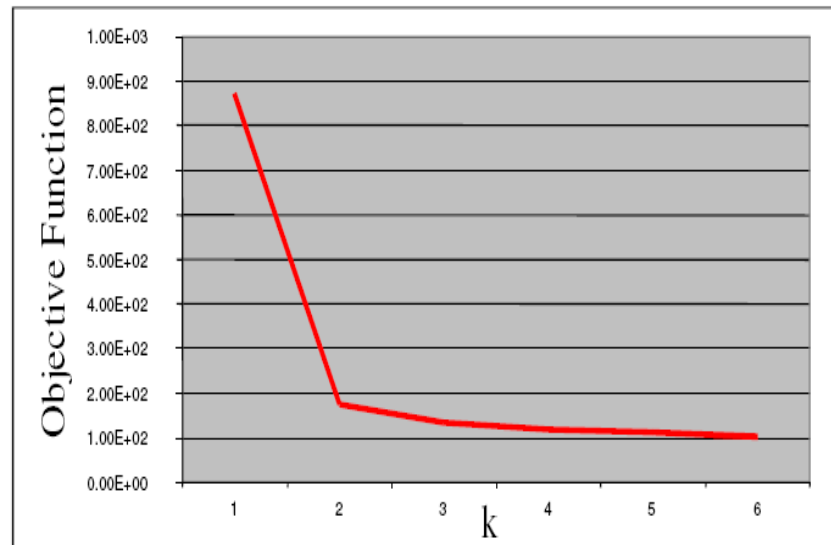


When $k = 3$, the objective function is 133.6



Deciding K

- We can plot objective function values for $k=1$ to 6
- The abrupt change at $k=2$ is highly suggestive of two clusters
- Known as “knee finding” or “elbow finding”
- Note that the results are not always as clear cut as in this toy example



Cluster Analysis

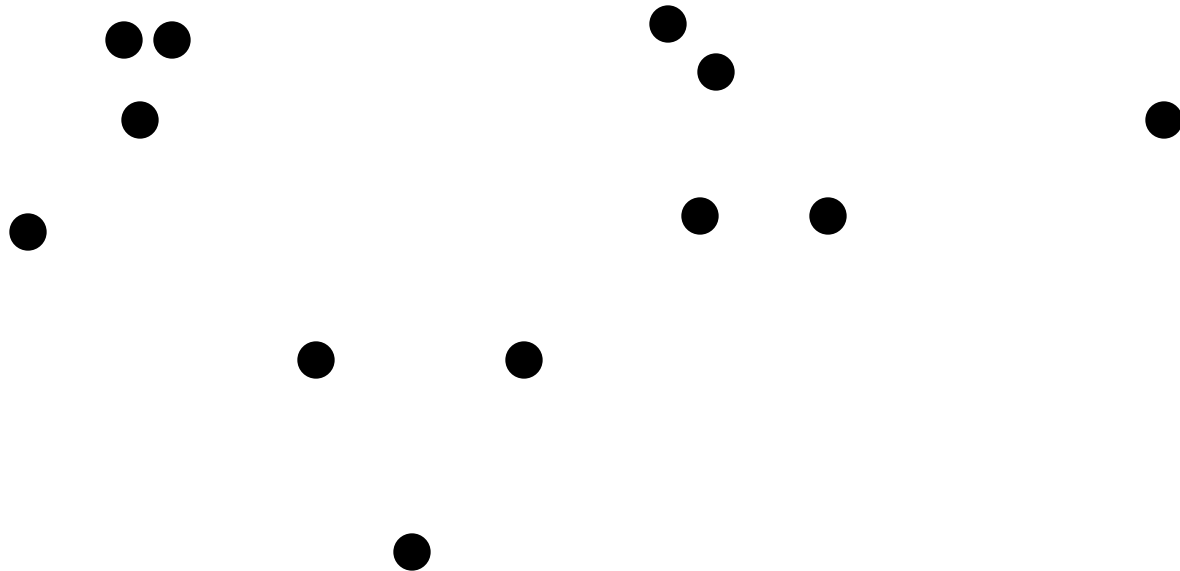
- Cluster Analysis: Basic Concepts
- Partitioning Methods
- Hierarchical Methods 
- Density-Based Methods
- Evaluation of Clustering
- Summary

Hierarchical clustering

- Agglomerative (Bottom-up)
 - This bottom-up strategy starts by placing each object in its own cluster
 - Merge the two most similar clusters into one
 - Replace the two clusters into the new cluster
 - Re-compute intercluster similarity scores w.r.t. the new cluster
 - Repeat the above step until there are k clusters left (k can be 1)
 - Most hierarchical clustering methods belong to this category. They differ only in their definition of intercluster similarity.

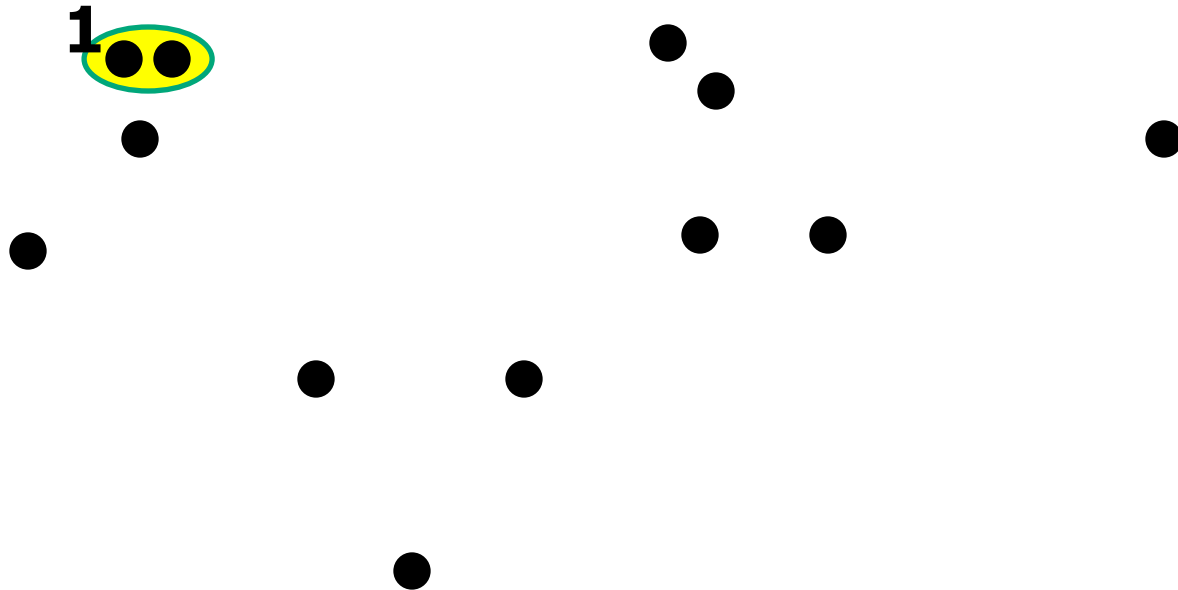
Agglomerative clustering

- Agglomerative (Bottom up)



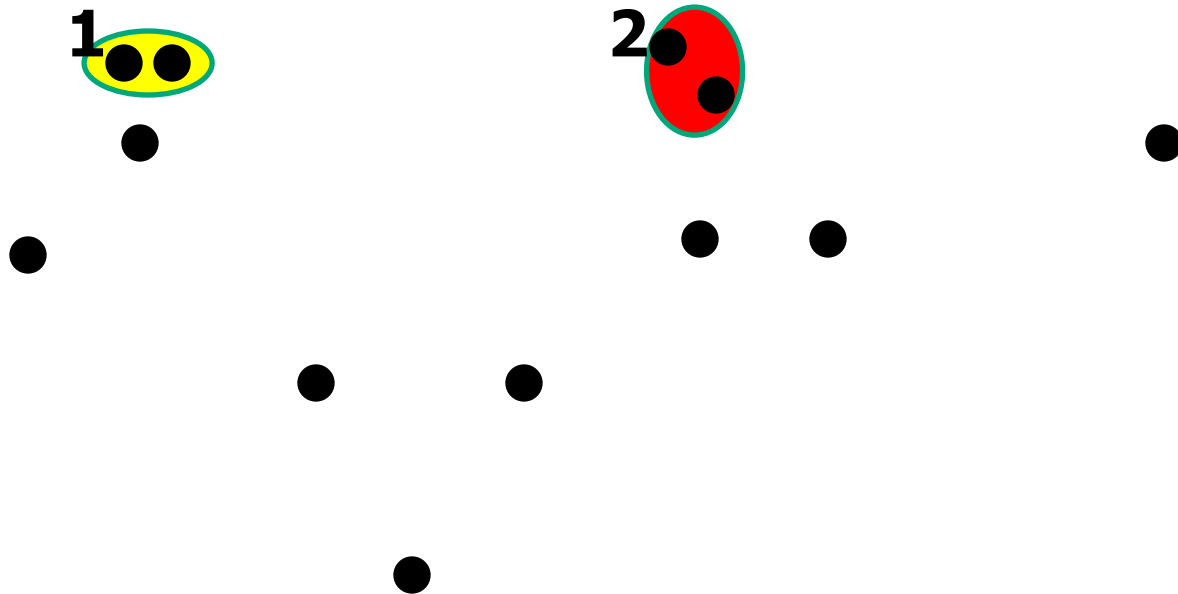
Hierarchical clustering

- Agglomerative (Bottom up)
- 1st iteration



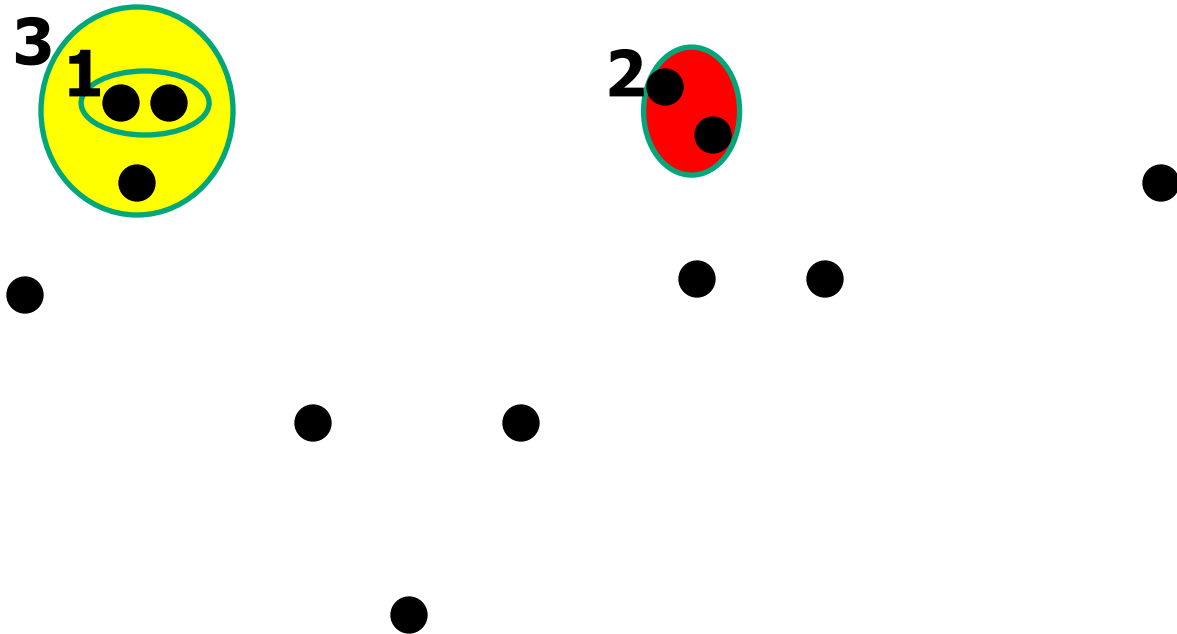
Hierarchical clustering

- Agglomerative (Bottom up)
- 2nd iteration



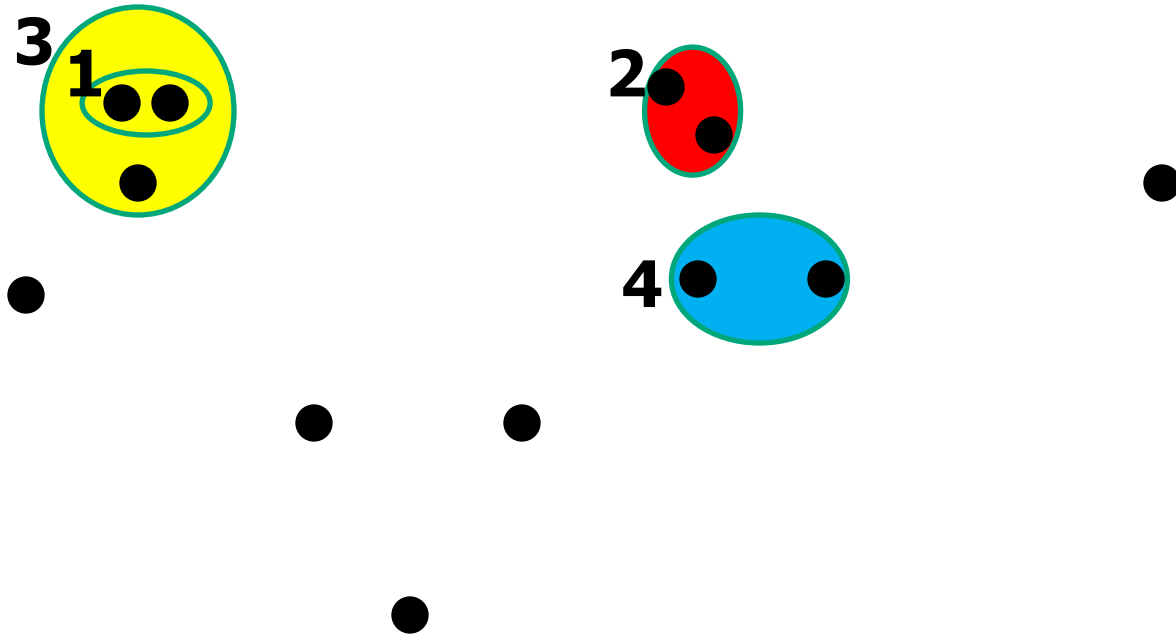
Hierarchical clustering

- Agglomerative (Bottom up)
- 3rd iteration



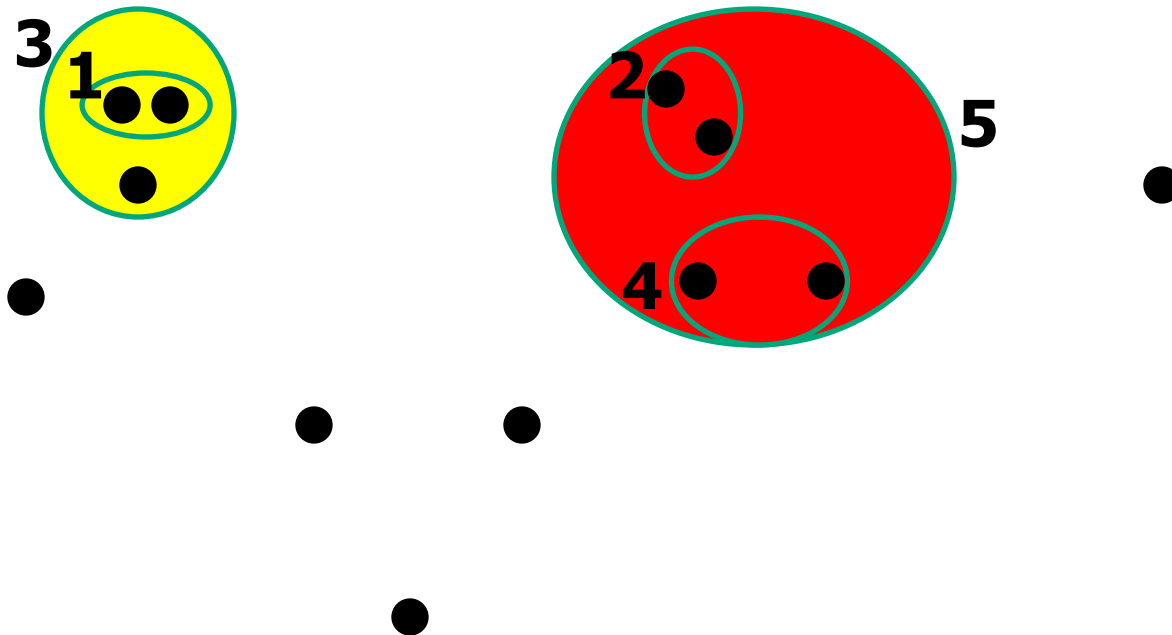
Hierarchical clustering

- Agglomerative (Bottom up)
- 4th iteration



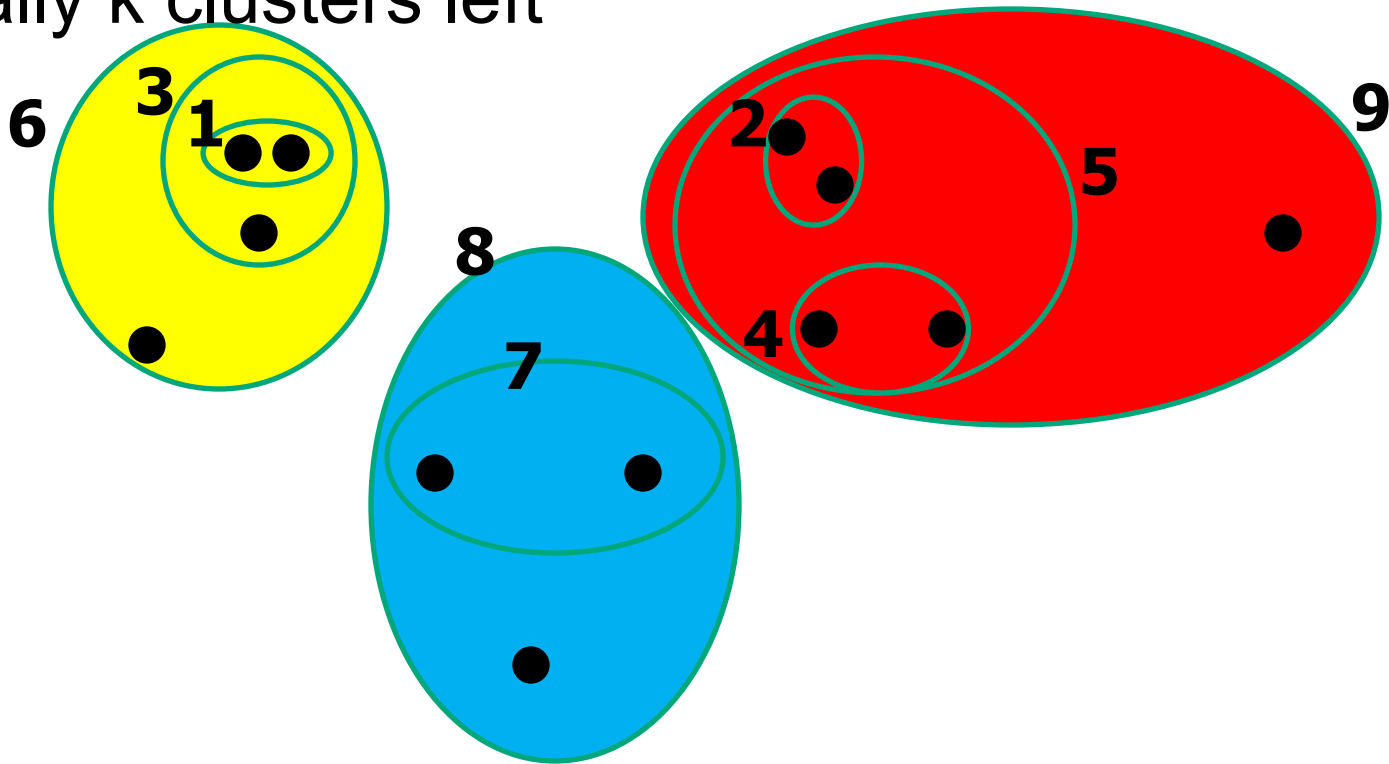
Hierarchical clustering

- Agglomerative (Bottom up)
- 5th iteration



Hierarchical clustering

- Agglomerative (Bottom up)
- Finally k clusters left



Intercluster similarity (δ)

- (1) **The single linkage** distance is the closest distance between two objects belonging to two different clusters.

$$\delta_1(S, T) = \min \left\{ d(x, y) \mid x \in S, y \in T \right\}$$

- (2) **The complete linkage** distance represents the distance between the most remote objects belonging to two different clusters.

$$\delta_2(S, T) = \max \left\{ d(x, y) \mid x \in S, y \in T \right\}$$

- (3) **The average linkage** distance defines the average distance between all the objects belonging to two different clusters.

$$\delta_3(S, T) = \frac{1}{|S| |T|} \sum_{\substack{x \in S \\ y \in T}} d(x, y)$$

Intercluster similarity (δ)

(4) **The centroid linkage** distance reflects the distance between the centers v_s and v_t of two clusters S and T respectively, defined as $\delta_4(S, T) = d(v_s, v_t)$, where

$$v_s = \frac{1}{|S|} \sum_{x \in S} x, \quad v_t = \frac{1}{|T|} \sum_{y \in T} y$$

(5) **The average of centroids linkage** represents the distance between the center of a cluster and all the objects belonging to a different cluster.

$$\delta_5(S, T) = \frac{1}{|S| + |T|} \left\{ \sum_{x \in S} d(x, v_t) + \sum_{y \in T} d(y, v_s) \right\}$$

Time Complexity of Agglomerative Clustering

- The time complexity of agglomerative clustering can vary depending on the chosen linkage criterion (e.g., single linkage, complete linkage, average linkage, etc).
- Here's a general overview of the time complexity:

(i) Computing Pairwise Distances

- Initially, we need to compute the pairwise distances between all data points.
- This step typically requires $O(n^2d)$ operations if we are using a simple distance metric like Euclidean distance, where d is the dimension of the object. More complex distance metrics may have higher time complexity.

Time Complexity of Agglomerative Clustering

(ii) Cluster Merging

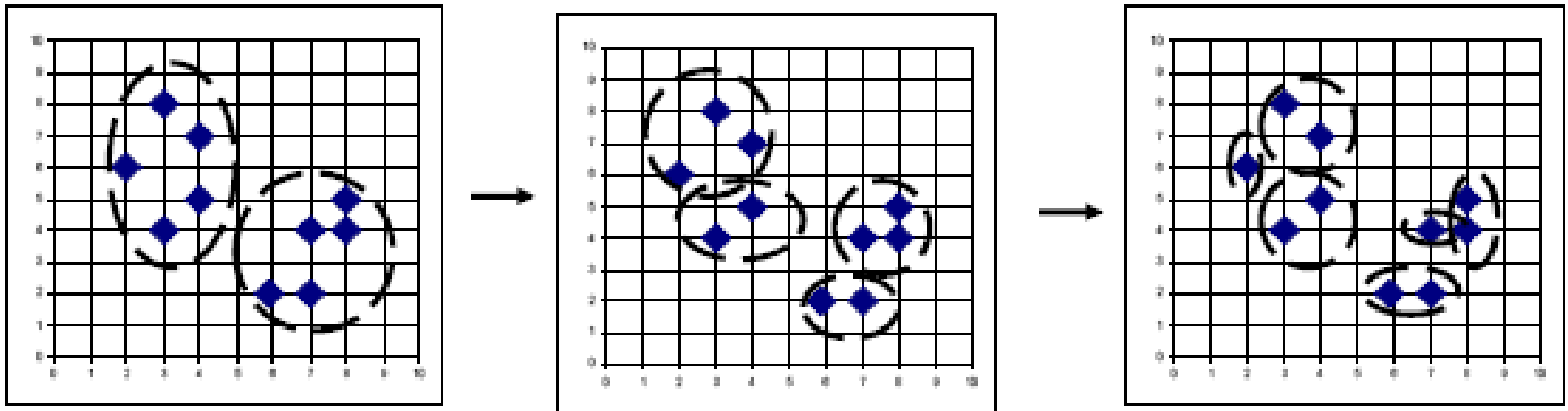
- At each iteration, we need to identify the closest pair of clusters and merge them.
- Identifying the closest pair of clusters can be done efficiently using data structures like priority queues or minimum spanning trees.
- Merging generally requires $O(n^2)$ operations if we use a brute-force approach to compare all pairs of clusters.
- Thus, the merging process requires $O(n^2)$ operations (to find the closest clusters) at each step, over $(n-1)$ merges.
- Thus, the overall time complexity for agglomerative clustering is: $O(n^3)$

Divisive clustering

- Divisive (Top-down)
 - Start at the top with all objects in one cluster
 - The cluster is split into subclusters using a flat clustering algorithm
 - This procedure is applied recursively **until** each object is in its own singleton cluster **or**
 - **until** it satisfies certain termination conditions, such as (i) a desired number of clusters is obtained or (ii) the distance between the two closest clusters is above a certain threshold distance.

Hierarchical clustering

- Divisive (Top-down): Use flat clustering say, K-Means clustering with $K=2$.



Time Complexity

- Divisive clustering is the opposite of agglomerative clustering, where we start with a single cluster containing all data points and recursively split it into smaller clusters.
- The time complexity of divisive clustering depends on the specific algorithm used for splitting clusters and the criteria for determining the optimal splits.
- Some divisive clustering algorithms may have exponential time complexity in the worst-case scenario, particularly if they exhaustively search for the best splits.
- It's important to note that these complexities are theoretical and can vary in practice based on the implementation details, optimizations, and characteristics of the dataset.
- Additionally, the time complexity may also be influenced by factors such as the dimensionality of the data and the choice of distance metric.

Time Complexity

1. Choosing a split criterion:

- In the divisive approach, the key part is how the algorithm decides where to split the data.

Common methods for splitting include:

- K-means (or other centroid-based methods): For each split, a clustering algorithm (like K-means) may be applied to the data points in the cluster. The time complexity of K-means clustering is $O(k \cdot n \cdot d)$, where k is the number of clusters, n is the number of data points, and d is the number of features.
- Other methods might involve calculating distances between points or minimizing some dissimilarity measure, which can also take significant time.

Time Complexity

2. Number of splits:

- The algorithm continues to split until there are the desired number of clusters. In the worst case, if the data is split into n clusters (where n is the number of data points), there will be $n - 1$ splits.

Overall Time Complexity:

- **Per split:** If you're using K-means (or a similar clustering approach) to split a cluster, the time complexity for a single split can be $O(k \cdot n \cdot d)$.
- **Number of splits:** There will be $O(n)$ splits if you are splitting until each data point is its own cluster.

Time Complexity

Thus, the **overall time complexity** for a divisive clustering algorithm is:

$$O(n^2 \cdot d) \quad (\text{in the worst case with a basic K-means splitting approach})$$

Where:

- n is the number of data points,
- d is the number of features,
- k is the number of clusters (typically assumed to be a constant or small).

If other, less expensive splitting criteria are used, or if the number of splits is reduced, the complexity might be better. However, for most general cases involving K-means-like splits, $O(n^2 \cdot d)$ is a reasonable upper bound.

Bottom-up vs. Top-down

- Which one is more complex?
- Which one is more efficient?
- Which one is more accurate?

Bottom-up vs. Top-down

- Which one is more complex?
 - Top-down
 - Because a flat clustering is needed as a “subroutine”
- Which one is more efficient?
- Which one is more accurate?

Bottom-up vs. Top-down

- Which one is more complex?
- Which one is more efficient?
- Which one is more accurate?

Bottom-up vs. Top-down

- Which one is more complex?
- Which one is more efficient?
 - Top-down
 - For a fixed number of top levels, using an efficient flat algorithm like K-means, divisive algorithms time complexity are $O(n^2)$, approximately
 - Agglomerative algorithms time complexity are $O(n^3)$, approximately

Bottom-up vs. Top-down

- Which one is more complex?
- Which one is more efficient?
- Which one is more accurate?

Bottom-up vs. Top-down

- Which one is more complex?
- Which one is more efficient?
- Which one is more accurate?
 - Top-down
 - Bottom-up methods make clustering decisions based on local patterns without initially taking into account the global distribution. These early decisions cannot be undone.
 - Top-down clustering benefits from complete information about the global distribution when making top-level partitioning decisions.

